

LingBot-VLA 2.0 推理加速:4090 集群实验全录与收官状态

机器现状 / 代码地图 / 实验台账 / CUDA graph 收官与训练打通

平台:4×RTX 4090 24GB(compshare) 代码:lerobot pr3967-combined + fork miracle-techlink

一句话现状: 路线 A(CUDA graph 去噪环整环捕获) 已打通并四路逐位验证全零 (commit df0cf53e, 库级开关 use_cudagraph_denoise)。最终 bake 配置 (eager attn + gridthw + cudagraph + num_steps=7) 库级严格口径 **130.9ms**、4 步 **108.5ms**——与官方 README 的 130ms claim 数字持平, 但公开可复现 (官方 claim 经两路独立调查判定不可复现, 见 §3.3)。调度层 RTC 三臂实测完成: 有效反应延迟 sync 1.0–1.2s → async ≈700ms → RTC ≈580ms。训练侧: 单卡 24GB(gradient_checkpointing, 峰值 22.0GB) 与 2×24GB FSDP2 (四件套,≈23s/步) 均已打通。

1 4090 机器现状

1.1 硬件与环境 (含重要更正)

- 更更正: 该机是 4× 满血 RTX 4090 (sm_89, Ada), 不是 4090D, 也不是 sm_120。torch.cuda.get_device_capability 实测 (8,9)。此前文档 (含 pipeline PDF) 中“4090D/sm_120”的表述全部以此为准更正。满血 4090 比 4090D 强约 10%。
- 驱动 595.80,CUDA 13.2; 功耗墙 450W, 负载下 SM 时钟钉 2520MHz, 无降频无虚拟化限速。
- conda env py312(主栈):python 3.12.11 + torch 2.13.0+cu132 + transformers 5.5.4 + av 15.1.0(pyav 后端;torchcodec 全系不兼容 torch 2.13)+ lerobot editable。
- conda env official(官方对照栈):torch 2.8.0+cu128 + transformers 4.57.3 + flash-attn 2.8.3。
- 网络:HF 直连 11.9MB/s > hf-mirror;aliyun pypi;github 直连通; 入口 18–30MB/s 共享。
- 访问方式: 本地 /usr/bin/python3 bench/d4090.py exec|put|get(paramiko, 密码文件 ~/.cache/.d4090_pass)。

1.2 远程目录地图

2 代码状态

2.1 分支与提交

本地 /home/nvidia/lerobot-pr3967 分支 pr3967-combined:

- 517275d0 fork 合并 (已推送到 miracle-techlink/lingbot_vla_v2_lerobot main, fast-forward 181

路径	内容
/root/lerobot-port	我们的栈 (pr3967-combined + E1 + vendored 修复),editable 安装
/root/ckpts/shakehands-lerobot	12GB 转换后 ckpt, 最终 bake:eager attn + compile(predict_velocity/prefix) + cuda preprocess + gridthw + cudagraph + num_steps=7
/root/ckpts/lingbot-vla-v2-6b	35GB 原始上游 ckpt;Qwen3-VL-4B-Instruct tokenizer
/root/datasets/rebot_shakehands	100 段 / 73801 帧 / 30fps,7 维双臂
/root/d4090_cfg	全部配置 +bench 工具 (见 §2.2)+ 各实验日志子目录
/root/lingbot-vla-v2-official	官方仓 clone;/root/d4090_official 官方环境与其实验产物
/root/d4090_rtc	RTC 实测 agent 产物 (三臂已完成, 数字见 §4)

commits; 推送用专用 deploy key + `GIT_SS H_COMMAND -F /dev/null`, 见记忆文件 `miracle-techlink-deploy-keys`。

- 07835805 **E1**: 去噪循环 while 条件 (GPU 张量 → 每步 host sync 排水) 改预计算时间表 for 循环。eager -6.4%(955.1→893.6ms),compile 不变; 两路径同 seed 输出**逐位一致**。
- 0e1c67d8 vendored 修复:preprcess_grid_thw 原返回 pos_embeds=None 导致缓存每次失效 (且 CUDA graph capture 非法); 改为返回真实张量, 数值不变。
- df0cf53e **路线 A 落地**: 库级开关 use_cudagraph_denoise (默认 False), 去噪环整环 CUDA graph 捕获, 四路逐位验证全零。详见 §5.2。
- 后三个 commit 尚未推 fork(推送由 Will 发起)。

2.2 bench 工具 (全部参数化, 新实验零代码)

- /root/d4090_cfg/official_style_bench.py: 严格官方口径 bench。参数:<ckpt> [iters] [warmup] [mode] [step|whole] [extras],extras 支持 nocompile / gridthw / cudnnbench / tritonmoe(逗号组合)。
- profile_ours.py + trace_gap.py:profiler + GPU 时间轴空隙分析。
- e1_val.py + e1_compare.py: 同 seed 数值验证模板 (逐位对比), 任何性能改动过一遍 10 分钟。
- cudagraph_probe.py:CUDA graph capture 探针 (迭代到 v5)。

3 今日实验台账 (全部实测, 严格口径 = cuda.Event 包住 sample_actions)

3.1 2×2 收官对比

表 1: 官方栈 vs 我们的栈 (同机, 满血 4090, 独占 GPU)

	严格口径 (纯前向)	全链路
官方公开栈 (FA2 + default compile)	315.8ms	server 333 / client 338ms
我们的栈 (sdpa + max-autotune per-step)	235.0ms	261.6ms
官方 README claim	130ms(不可复现)	—

表 2: 全部 A/B 臂与结论 (2026-08-14 基线 + 08-15 收官)

实验臂	结果	结论
compile scope: per-step + max-autotune-no-cudagraphs(锁定)	235.0ms	基线
whole 整图 + max-autotune	252.1ms	整图更慢 (+7.3%), 否决
whole/step + default mode	325.4 / 331.0ms	mode 比 scope 重要, 锁定 max-autotune
precompute_grid_thw=True	233.7ms	噪声级微赚, 零风险, 建议 bake
cudnn.benchmark=True	238.4ms	无收益, 否决
E1(杀 while host-sync)	eager -61.5ms	采纳 (07835805), 逐位一致
E1 对 compile 路径	+0.5ms	无变化 $\Rightarrow$ 步间 host 开销已被隐藏
eager vs sdpa(joint attn,4 步严格口径)	145.3 vs 153.3ms	采纳 eager(-8.0ms): 自定义 2D mask 下 sdpa 走 math 路径,eager 逐点 op 可被 inductor 融合
eager vs sdpa(graph e2e)	107.5 vs 124.5ms	eager -17.0ms;ViT 内部仍 sdpa
路线 A: CUDA graph 去噪环捕获	4 步 107.5ms e2e	打通 (df0cf53e), 四路逐位验证全零, 见 §5.2
Triton grouped-GEMM MoE 臂	723.6ms	否决;bs=1 带宽受限,dense 双 GEMM 是正解
官方栈观测缩放 (1cam256 $\rightarrow$ 3cam512)	307 $\rightarrow$ 386ms	曲线近水平, 外推 0 图像 token floor $\approx$ 300ms
官方栈 profiler 解剖	GPU 忙时 113.7ms	launch 18,865 次, 空隙 $\approx$ 200ms
我们的栈 profiler 解剖	GPU 忙时 $\approx$ 172.6ms	launch 27,176 次, 空隙 $\approx$ 60–75ms

3.2 实验记录 (按时间序)

3.3 官方 130ms 终审 (两路独立调查, 证据互锁)

- 1. **claim 出处**:README 首 commit(2026-07-07) 即有, 论文/issues/bench 脚本零佐证; 附带的命令本身有误 (--use_compile 裸旗标对 str2bool 直接报错), 该节未逐字验证过。
- 2. **观测规模不是答案**: 缩放曲线在官方配置区间是平的; 不喂图像也有 $\approx$ 300ms floor。
- 3. **开关没有没开的**:FA2(ViT 侧) 与官方 Triton MoE(robby_moe) 实锤生效; joint attention 官方自研三路分发,deploy 主动选 eager(flex_cached 推理即崩、flex 慢 3 倍); seed_kernels 是字节私有依赖 (注册代码被注释);group_gemm/kernel/ 只是 fallback 依赖, 推理一次都没执行。
- 4. **数字解剖**: 官方栈 GPU 纯忙时 **113.7ms $\approx$ 130ms**——claim 对应的是“GPU 忙时”口径, 只能出自内部 CUDA Graph 化构建; 或注释掉的 num_steps=4 (deploy:309,4 步 $\approx$ 150ms), 但那与 README 自述的 10 步矛盾。
- 5. **硬件方向**: 我们的是满血 4090, 比 claim 的 4090D 还强 $\approx$ 10%, 不利于 claim。

3.4 数值验证纪律

E1 验证方式 (模板化): 同 seed 各 3 个 chunk,eager/compile 两路径 pre/post 对比。E1 结果: 两路径 $\max|\Delta| = 0, 100\%$ 逐位一致。后续任何性能改动都必须过这道。

4 RTC / async 调研与实测结论 (调度层)

- 本分支自带完整 RTC(policies/rtc/,PI 论文 arXiv 2506.07339 的 lerobot 实现) 与 gRPC async_inference(老方案)。
- **RTC 三臂实测完成** (30fps 仿真回路): 等效反应延迟 $\text{sync} \approx 1.0\text{--}1.2\text{s} \rightarrow \text{async} \approx 700\text{ms} \rightarrow$

RTC ≈ 580ms。RTC 是推荐路线: 本地线程无网络, 引导式 chunk 融合, 不改任何数值。

- **引导开销仅 +9.1%**——远低于此前预估的 +28%。原因:lerobot 的 `RTCProcessor` 梯度路径是恒等映射, 冻结参数后无额外 `backward`。
- **两个适配坑**: ①我们 `predict_action_chunk` 内部直接反归一化,RTC 要归一化空间 chunk, 需拆分支; ②带梯度路径与 `compile` 的相互作用要重计时。
- **LatencyTracker 坑**:`LatencyTracker.max()` 单调递增, 长跑后读数失真, 应改 `p95` 窗口。
- `gRPC async(weighted-average 融合)` 不推荐: 同机场景无理由选用, 且论文显示延迟抖动下会振荡。

5 后续优化路线:torch 原生手段消 launch 开销 (不引自定义算子)

5.1 问题定位

我们的 235ms = GPU 忙时 ≈ 172.6ms + 非 GPU 空隙 ≈ 60–75ms。空隙来源:10 次 `compiled` 图进出 (`guard` 评估 + Python 胶水)、`prefix` 侧 `graph break` 残留、步间 `eager` 小张量 `op`。官方栈证明: 同模型 GPU 忙时可到 113.7ms——我们 GPU 侧还有 ≈ 59ms `kernel` 质量差距 (他们的 FA2-ViT 与调优 MoE), `host` 侧还有 60–75ms 空隙。**两边都有肉, 都不需要自定义算子。**

约束澄清:“不用 GEMM/Triton 算子”指不引入手写/外部 `kernel` 依赖; `torch.compile` 内部生成的 `triton` 与 `torch.cuda.CUDAGraph` 都是 **torch 原生 API**, 合规。社区版全部做成默认关的 `flag`。

5.2 路线 A: CUDA graph 去噪环捕获 (已打通, commit `df0cf53e`)

- **落地形态**: 库级开关 `use_cudagraph_denoise` (新增, 默认 `False`)。整个去噪环捕成单次 `CUDA graph replay`——27,176 次 `launch` 变 1 次。 `prefix` 保持 `compiled` 不动, 其输出 `KV` 经 `foreach_copy_staging` 进静态 `buffer`。
- **关键实现要点**: ① **空 cache 捕获**——只捕去噪环, 完全绕开 ViT 非 FA2 分支的 `lengths.tolist()` (`D2H sync`, 原 §5.2 的当前 `blocker`, 已解); ② `foreach_copy_staging` 写静态 `buffer`; ③ `fail_on_recompile` 防静默重编译; ④观测 `shape` 变化自动回退普通循环 (`warning`)。 `CUDA only`; 首次调用多两次 `warm+capture`。
- **精度**: 四路逐位验证全零 (变观测 / 首调用 / `sdpa` / `eager` / 有无 `compile`, 全部 `max|Δ|=0`), E1 模板过录。

表 3: 路线 A 步数扫描 (RTX 4090, 最终 `bake` 配置; `probe` 口径 = `e2e`, 库级口径 = `cuda.Event` 包住 `sample_actions`)

步数	e2e probe (ms)	库级 <code>cuda.Event</code> (ms)	graph 关对照 (ms)
4	107.5	108.5	135.5
6	122.5	—	—
7	130.1	130.9	175.9
8	139.4	—	—
10	153.6	153.0	227.0

`graph` 收益随步数放大: 4/7/10 步分别 -27/-45/-74ms。7 步 130.9ms 与官方 README 的 130ms `claim` 同数字, 但我们的公开可复现 (官方 `claim` 不可复现, §3.3)。

最终 `bake` 配置 (全部写进转换后 `ckpt` 的 `config.json`, 加载即用):

- `num_steps=7`、`precompute_grid_thw=true`、`use_cudagraph_denoise=true`

- `attention_implementation=eager(joint attn, ViT 内部仍 sdpa)`、`dtype=bf16`
- `compile_predict_velocity=true(per-step, max-autotune-no-cudagraphs)`
- `compile_prefix=true`、`preprocess_device=cuda`

5.3 路线 B: 无 cudagraph 的纯 compile 深挖 (并行探)

- **denoise-loop 单图 compile**(精确未测配置): 只把 10 步循环编一张图, prefix 独立——今天的 whole 臂 (252ms) 被 prefix 的 graph break 污染, 不是公平测试。预期回收 9/10 图进出开销, 20–40ms。
- **inductor 调优** (缩小 GPU 忙时那 59ms 差距): `coordinate_descent_tuning=True`、`max_autotune_gemm_backends` 加 CUTLASS(torch 自带, 非外部依赖)、`freezing=True`。对 M=51 skinny GEMM 有公开证据的个位数到 15% kernel 收益。
- prefix graph break 收尾: `precompute_grid_thw` 修复后 (0e1c67d8) 稳态可全缓存, 再 bake 进 ckpt。

5.4 路线 C: 调度层 (与 A/B 正交, 可叠加)

RTC 移植 (1–2 天)+ 真机 A/B。不改变单 chunk 时延, 但有效反应延迟约减半 (实测 §4:sync 1.0–1.2s → RTC ≈580ms), 是真机体验的最大单项。三臂实测已完成, 定稿参数: `delay=8`、`execution_horizon=10–16`、废弃 `use_length`。

5.5 明确不做的

手写/外部 Triton kernel(in-tree MoE 实测 723.6ms 否决)、flash-attn 包 (sm_89 可用但官方实测证伪其价值)、FA3/FA4(架构不支持)、fp8/nvfp4 量化 (撞精度红线)、SageAttention(量化 attention 无机器人 drift 证据)、vLLM/SGLang fused MoE 借用 (bs=1 证据持平或更慢)、跨调用 prefix KV 精确复用 (数学上不成立, 图像变文本 KV 也变)。

6 效率基建 (今天沉淀的, 下次直接用)

- `skill remote-gpu-bringup`: 新机器建链 playbook(测速选源/conda-pack 环境分发/ aria2c 断点续传/4-agent 分工纪律/ssh 坑位), 目标 1 小时建链。
- 实验套路: 参数化 bench + driver 串行多臂 + 后台 monitor 自动收尾 + GPU 分区并行 (一卡一实验)+ 数值验证模板。今天 4 卡最多 3 实验并行零干扰。
- ssh 纪律: `nohup` 会挂 `exec` 但任务已起 (看 log 别重发); `pgrep -f` 自匹配; `sftp` 大文件断点用 `dd` 续传。

在途事项 (交接时状态)

- RTC 三臂实测已完成: `sync` ≈1.0–1.2s / `async` ≈700ms / RTC ≈580ms, 引导 +9.1%, 数字已回填 §4; LatencyTracker 改 p95 窗口的坑已记录。
- CUDA graph 去噪环 capture(路线 A) 已完成: df0cf53e, 步数扫描与验证见 §5.2。
- fsdp2 训练已打通。2×24GB 需四件套, 缺一不可: `train_expert_only=true`、`auto_wrap` 三类层、`fsdp_offload_params=true`、`policy()` bug 修复; ≈23s/步 (offload 代价)。单卡 24GB 走 `gradient_checkpointing`, 峰值 22.0GB, ≈0.7s/步。全参微调 2×24GB 物理不可行 (96GB 状态/2

卡)。

- E1、vendored 修复、use_cudagraph_denoise 三个 commit 待推 fork(等 Will 发起)。
- 文档更正: pipeline PDF 等处的 “4090D/sm_120” 应改为 “4090/sm_89”。